

Evaluation Java 2

1) Analyse technique

a. Points positifs :

- Architecture peu complexe
- Simple à déployer
- Utilisation de Spring Boot et Spring Data JPA : encadre le projet dans des normes prédéfini par le framework
- Pas de latence réseau entre les couches

b. Points négatifs :

- couplage logique métier / contrôler : peu flexible pour des évolutions, pas de test unitaire
- Rendu coté serveur : pas de rendu en « temps réel »
- Aucune Api REST : difficile de brancher une app mobile
- Front et Back non séparé : complexifie le développement en parallèle
- JS minimal sans framework : augmente la complexité et la difficulté pour maintenir une app UI réactive. Et si changement de développeur difficulté de recrutement accrue
- Authentification session HTTP : inadapté au mobile, ne permet pas de scalabilité (même si pas forcément nécessaire pour le contexte de FoodLink)

2) Architecture proposée

Backend API REST Spring Boot + Frontend séparé (Angular/React)

Authentification JWT stateless → compatible mobile et scalable

Séparation claire Controller/Service/Repository

Évolution possibles :

un service de notifications si on veut alerter les clients par SMS/email quand leur commande est prête

un service de reporting/analytics séparé si FoodLink veut analyser les données sans impacter la prod

Cette architecture répond aux faiblesses identifiées.

La séparation en couches Controller/Service/Repository découple la logique métier des contrôleurs, facilitant les tests unitaires et les évolutions futures.

L'exposition d'une API REST permet de brancher n'importe quel client, application web, mobile ou autre sans modifier le backend.

Le remplacement de l'authentification par session HTTP par un système JWT stateless rend l'application compatible mobile et modernise l'application.

Enfin, la séparation du frontend dans un projet indépendant permet aux équipes back et front de travailler en parallèle sur la base d'un contrat d'API défini.